ESP32 Lab

Beginner to Real-World — IoT Fun Codes

What is the ESP32?

A low-cost microcontroller with built-in Wi-Fi and Bluetooth — dual-core 240MHz, 520KB RAM, and enough GPIOs to interface with almost any sensor or peripheral. Think of it as a tiny computer that can sense, compute, and communicate wirelessly on a single chip.

Board	Notes
ESP32 DevKit	38 pins, more GPIOs, dual-core 240MHz
NodeMCU ESP32-S	Narrower, breadboard-friendly, same chip

GPIO Pin Layout

Pin Types — Quick Reference

Type	Pins	Use
Digital I/O	Most GPIOs	LED, button, relay, sensor ON/OFF
Analog (ADC)	GPIO 32-39	Read voltage — sensors, potentiometers
PWM	Almost any GPIO	Dim LEDs, servo motors
DAC	GPIO 25, 26	True analog voltage output
I2C	GPIO 21 (SDA), 22 (SCL)	OLED, BMP280, MPU6050
SPI	GPIO 18/19/23/5	SD cards, TFT displays
UART	GPIO 1 (TX), 3 (RX)	Serial comms, GPS, Bluetooth serial
Touch	GPIO 4,13,14,15,27,32,33	Capacitive touch buttons
Strapping	GPIO 0, 2, 12, 15	Affect boot — avoid at startup

GPIO 34-39 are input-only (no pull-up). GPIO 6-11 are tied to onboard flash — do NOT use.

Wi-Fi Operating Modes

STA Mode (Station)	ESP32 connects to your home router like any phone or laptop. Gets an IP via DHCP. Use for cloud data, NTP time sync, and web API calls.
AP Mode (Access Point)	ESP32 becomes the router — creates its own Wi-Fi network. Fixed IP: 192.168.4.1. No internet needed. Connect phone to its network, open browser, done.
AP + STA (Dual Mode)	Both simultaneously. ESP32 stays connected to the internet while also hosting its own AP. Useful as a bridge device or for a local dashboard while staying cloud-connected.

Sketch Index

#	Folder	What it does	New concept
01	blink_led	Toggle onboard LED	GPIO, digital write
02	get_mac	Read device MAC address	Device identity, Serial
03	wifi_sta_mode	Connect to router, print IP	STA, RSSI, reconnect
04	wifi_ap_mode	ESP32 hosts its own Wi-Fi	AP mode, 192.168.4.1
05	ap_sta_dual_mode	AP and STA simultaneously	WIFI_AP_STA, bridge
06	web_server_led	Control LED from browser	HTTP server, HTML over Wi-Fi
07	dht_sensor	Live temp and humidity, JSON endpoint	External sensor, JSON
08	esp_now_peer	Two boards talk, no router	ESP-NOW, struct packets
09	oled_i2c	OLED: IP, signal, uptime	I2C, Adafruit GFX
10	mqtt_broker	Publish data, receive commands	MQTT, pub/sub, Home Assistant

---

Language & Toolchain Options

All sketches use **C++ (Arduino-flavoured)**. setup() and loop() are entry point conventions wrapping Espressif's ESP-IDF. Alternatives below:

Language	Tool	Good for	Tradeoff
C++ (Arduino)	Arduino IDE, PlatformIO	Beginners, huge libraries	Abstracted, less control
C/C++ (ESP-IDF)	VS Code + IDF plugin	Full control, production firmware	Steep curve, verbose
MicroPython	Thonny, mpremote	Python familiarity, prototyping	Slower, less RAM
CircuitPython	Mu Editor	Educational, drag-drop FS	Limited ESP32 support
Lua	NodeMCU firmware	Event-driven scripting	Legacy, less maintained
JavaScript	Moddable SDK, Espruino	JS familiarity	Niche, small community
Rust	esp-hal, esp-idf-hal	Memory safety, modern tooling	Experimental setup

---

Coming in Batch 3

11	**WiFiManager**	Captive config portal — no hardcoded credentials

12	**NTP time sync**	Real clock time without RTC hardware
13	**Google Sheets**	HTTP POST data logging to a spreadsheet
14	**BLE scan**	Detect nearby Bluetooth devices
15	**Deep sleep**	Wake on timer — battery-powered sensor node

---